

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №6
по дисциплине: БД
тема: «Организация взаимодействия с базой данных через приложение с
графическим интерфейсом»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Панченко М. В.

Белгород 2025

Цель работы: получить навыки подключения к различным системам управления базами данных и взаимодействия с ними. Разработать консольное приложение для взаимодействия с базой данных.

Задание к работе:

Вариант 14

1. Изучить библиотеку для реализации приложения с графическим интерфейсом на выбранном языке программирования.
2. Разработать приложение с графическим интерфейсом, которое обеспечит подключение к базе данных, разработанной на основе предыдущих лабораторных работ, а также обеспечит выполнение запросов.

Структура моей базы данных из предыдущих работ

```
railway=# \dt
```

Список отношений			
Схема	Имя	Тип	Владелец
public	car	таблица	postgres
public	contactdetails	таблица	postgres
public	passenger	таблица	postgres
public	route	таблица	postgres
public	schedule	таблица	postgres
public	seat	таблица	postgres
public	ticket	таблица	postgres
public	ticketstatus	таблица	postgres
public	train	таблица	postgres

(9 строк)

Код консольного приложения для взаимодействия с БД:

```
import sys
import psycopg2
from typing import List, Tuple
from PyQt6.QtWidgets import (
    QApplication,
    QMainWindow,
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QPushButton,
    QLabel,
    QLineEdit,
    QTableWidgetItem,
    QTableWidgetItem,
    QMessageBox,
    QGridLayout,
    QScrollArea,
)
from PyQt6.QtCore import Qt
```

```
DB_CONFIG = {
    "host": "localhost",
    "database": "railway",
    "user": "postgres",
    "password": "postgres",
    "port": 5432,
}
```

```
class DatabaseService:
```

```

def __init__(self):
    try:
        self.connection = psycopg2.connect(**DB_CONFIG)
        self.cursor = self.connection.cursor()
    except psycopg2.Error as e:
        raise ConnectionError(f"ошибка подключения к бд: {e}")

def execute_query(self, query: str) -> List[Tuple]:
    try:
        self.cursor.execute(query)
        return self.cursor.fetchall()
    except psycopg2.Error as e:
        print(f"ошибка выполнения запроса: {e}")
        return []

def close(self):
    if self.cursor:
        self.cursor.close()
    if self.connection:
        self.connection.close()

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("система железнодорожных касс")
        self.resize(1000, 700)
        self.db = DatabaseService()

        central_widget = QWidget()
        self.setCentralWidget(central_widget)
        layout = QVBoxLayout(central_widget)

        title = QLabel("система железнодорожных касс")
        title.setStyleSheet("font-size: 20px; font-weight: bold; margin: 15px;")
        title.setAlignment(Qt.AlignmentFlag.AlignCenter)
        layout.addWidget(title)

        selector_layout = QHBoxLayout()
        selector_layout.addWidget(QLabel("выберите запрос"))
        self.combo = QComboBox()
        self.combo.addItem(
            [
                "билеты по типам поездов",
                "средняя цена по маршрутам",
                "количество мест по типам вагонов",
                "популярные направления по сезонам",
                "рейтинг поездов по типам вагонов",
                "поезда из а в б (ввод станций)",
            ]
        )
        self.combo.currentIndexChanged.connect(self.on_query_changed)
        selector_layout.addWidget(self.combo)
        layout.addLayout(selector_layout)

```

```

self.input_layout = QGridLayout()
self.input_layout.addWidget(QLabel("станция отправления"), 0, 0)
self.dep_input = QLineEdit()
self.input_layout.addWidget(self.dep_input, 0, 1)
self.input_layout.addWidget(QLabel("станция прибытия"), 1, 0)
self.arr_input = QLineEdit()
self.input_layout.addWidget(self.arr_input, 1, 1)
layout.addLayout(self.input_layout)
self.input_layout.setEnabled(False)

self.run_button = QPushButton("выполнить запрос")
run_style = "font-size: 14px; padding: 10px; background-color: #4a90e2;
color: white; border: none; border-radius: 5px;"
self.run_button.setStyleSheet(run_style)
self.run_button.clicked.connect(self.execute_current_query)
layout.addWidget(self.run_button)

scroll = QScrollArea()
self.table = QTableWidgetItem()
self.table.setEditTriggers(QTableWidgetItem.EditTrigger.NoEditTriggers)
self.table.setAlternatingRowColors(True)
self.table.setStyleSheet("font-size: 13px;")
scroll.setWidget(self.table)
scroll.setWidgetResizable(True)
layout.addWidget(scroll, 1)

self.status = QLabel("готово к работе")
layout.addWidget(self.status)

def on_query_changed(self, index):
    self.input_layout.setEnabled(index == 5)
    if index == 5:
        self.status.setText("введите станции и нажмите выполнить")
    else:
        self.status.setText("нажмите выполнить запрос")

def execute_current_query(self):
    query_index = self.combo.currentIndex()
    self.status.setText("выполняется запрос...")
    QApplication.processEvents()

    try:
        if query_index == 0:
            self.query_tickets_by_train_type()
        elif query_index == 1:
            self.query_avg_price_by_route()
        elif query_index == 2:
            self.query_seats_by_car_type()
        elif query_index == 3:
            self.query_popular_routes_by_season()
        elif query_index == 4:
            self.query_train_rating_by_car_type()
        elif query_index == 5:
            self.query_trains_from_a_to_b()
    except Exception as e:

```

```

        QMessageBox.critical(self, "ошибка", f"произошла ошибка\n{e}")
        self.status.setText("ошибка при выполнении")

def show_results(self, headers: List[str], rows: List[Tuple]):
    self.table.clear()
    if not rows:
        self.table.setRowCount(1)
        self.table.setColumnCount(1)
        self.table.setItem(0, 0, QTableWidgetItem("данные отсутствуют"))
        self.status.setText("данные отсутствуют")
        return

    self.table.setColumnCount(len(headers))
    self.table.setHorizontalHeaderLabels(headers)
    self.table.setRowCount(len(rows))

    for i, row in enumerate(rows):
        for j, value in enumerate(row):
            item = QTableWidgetItem(str(value))
            self.table.setItem(i, j, item)

    self.table.resizeColumnsToContents()
    self.status.setText(f"загружено записей: {len(rows)}")

def query_tickets_by_train_type(self):
    query = """
    SELECT t.TrainType, COUNT(tk.ID) as TotalTickets
    FROM Train t
    LEFT JOIN Schedule s ON t.ID = s.TrainID
    LEFT JOIN Car c ON s.ID = c.ScheduleID
    LEFT JOIN Seat st ON c.ID = st.CarID
    LEFT JOIN Ticket tk ON st.ID = tk.SeatID
    GROUP BY t.TrainType;
    """

    results = self.db.execute_query(query)
    self.show_results(["тип поезда", "всего билетов"], results)

def query_avg_price_by_route(self):
    query = """
    SELECT r.DepartureStation, r.ArrivalStation,
           ROUND(AVG(tk.Price)::numeric, 2) as AveragePrice
    FROM Route r
    JOIN Schedule s ON r.ID = s.RouteID
    JOIN Car c ON s.ID = c.ScheduleID
    JOIN Seat st ON c.ID = st.CarID
    JOIN Ticket tk ON st.ID = tk.SeatID
    GROUP BY r.DepartureStation, r.ArrivalStation;
    """

    results = self.db.execute_query(query)
    self.show_results(["откуда", "куда", "средняя цена"], results)

def query_seats_by_car_type(self):
    query = """
    SELECT c.CarType, SUM(c.TotalSeats) as TotalSeatsSum
    FROM Car c
    """

```

```

        JOIN Schedule s ON c.ScheduleID = s.ID
        JOIN Route r ON s.RouteID = r.ID
        GROUP BY c.CarType;
        """

        results = self.db.execute_query(query)
        self.show_results(["тип вагона", "всего мест"], results)

def query_popular_routes_by_season(self):
    query = """
    SELECT
        CASE
            WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (12, 1, 2) THEN
'зима'
            WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (3, 4, 5) THEN
'весна'
            WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (6, 7, 8) THEN
'лето'
            WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (9, 10, 11) THEN
'осень'
        END AS сезон,
        r.DepartureStation,
        r.ArrivalStation,
        COUNT(t.ID) AS пассажиров
    FROM Ticket t
    JOIN Seat st ON t.SeatID = st.ID
    JOIN Car c ON st.CarID = c.ID
    JOIN Schedule s ON c.ScheduleID = s.ID
    JOIN Route r ON s.RouteID = r.ID
    GROUP BY сезон, r.DepartureStation, r.ArrivalStation
    ORDER BY сезон, пассажиров DESC;
    """

    results = self.db.execute_query(query)
    self.show_results(["сезон", "откуда", "куда", "пассажиров"], results)

def query_train_rating_by_car_type(self):
    query = """
    SELECT
        tr.TrainNumber,
        tr.TrainType,
        c.CarType,
        COUNT(t.ID) AS продано
    FROM Ticket t
    JOIN Seat st ON t.SeatID = st.ID
    JOIN Car c ON st.CarID = c.ID
    JOIN Schedule s ON c.ScheduleID = s.ID
    JOIN Train tr ON s.TrainID = tr.ID
    GROUP BY tr.TrainNumber, tr.TrainType, c.CarType
    ORDER BY c.CarType, продано DESC;
    """

    results = self.db.execute_query(query)
    self.show_results(
        ["номер поезда", "тип поезда", "тип вагона", "продано билетов"],
results
    )

```

```

def query_trains_from_a_to_b(self):
    departure = self.dep_input.text().strip()
    arrival = self.arr_input.text().strip()

    if not departure or not arrival:
        QMessageBox.warning(self, "внимание", "введите обе станции")
        self.status.setText("ошибка: не указаны станции")
        return

    query = f"""
    SELECT * FROM get_trains_from_a_to_b('{departure}', '{arrival}');
    """
    results = self.db.execute_query(query)
    self.show_results(
        ["номер поезда", "тип поезда", "отправление", "прибытие"], results
    )

def closeEvent(self, event):
    self.db.close()
    event.accept()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

```


Результат работы:

система железнодорожных касс

выберите запрос

поезда из а в б (ввод станций)

станция отправления

Москва

станция прибытия

Казань

выполнить запрос

номер поезда	тип поезда	отправление	прибытие	
1	0025	пассажирский	2025-06-02 09:00:00	2025-06-02 18:00:00

Вывод: В процессе выполнения лабораторной работы были получены навыки подключения к различным системам управления базами данных и взаимодействия с ними. Так же мы смогли реализовать свой собственный графический интерфейс для работы с базой данных